

一元导数与局部变化实验

目标与输入

本实验用一个可手算的单参数模型，把解析导数、精确分数差商和浮点差分放在同一条核对链上。完成后应能解释成功核对、不可导反例以及步长限制。

输入仅为随附的 `data/calibration.csv`。三条无量纲合成记录为 $(x, y) = (1, 2), (2, 3), (3, 5)$ ，预测为 wx ，损失为平均平方误差。没有联网数据、真实个人记录、付费接口或随机抽样，也不评价真实任务表现。

`experiment.py` 的核心只依赖 Python 标准库；`experiment.ipynb` 逐步调用它，并使用 Matplotlib 画误差曲线。请保留整个单元目录，尤其不要只移动 Notebook 而遗漏脚本或 data 文件夹。

一 建立环境并从头运行

若前面课程的 Python 环境已经可用，可以继续使用。实际验收为 Python 3.12.14；Notebook 绘图使用 Matplotlib 3.10.8。第一次安装依赖需要网络，之后本实验完全离线。

可在 Anaconda Prompt 或终端创建一个环境：

```
conda create -n ml013 python=3.12 matplotlib jupyterlab
conda activate ml013
jupyter lab
```

以上是安装建议，请使用官方来源并记录实际版本；标准库脚本不需要 Matplotlib。

在本目录打开 Notebook，重新启动内核并运行全部单元。首格报告版本与文件是否存在；不要跳过错误去读取旧输出。

本包的 Notebook 已在新的 Python 进程中通过真实 ipykernel 进程内内核顺序运行。浏览器 UI、外进程 socket 传输以及学习者本机 Anaconda 安装未测试；这与代码已经执行是不同的验证范围。

在本单元目录可独立执行 `python experiment.py`，默认结果写入 `outputs`。另存时运行 `python experiment.py --output my_results`。下一次运行会替换同目录结果；保留比较实验时使用不同目录。输入路径相对于脚本，从其他工作目录调用也应成功。

二 先手算 再用程序核对

固定 $w = 3/2$ ，先在纸上列出预测、残差和平方误差，不要直接运行完整脚本。

输入 x	目标 y	预测 wx	残差 $wx-y$	残差平方
1	2	$3/2$	$-1/2$	$1/4$
2	3	3	0	0
3	5	$9/2$	$-1/2$	$1/4$

平均损失为 $1/6$ 。逐项导数贡献是 $2x_i(wx_i - y_i)/3$ ，三项分别为 $-1/3, 0, -1$ ，总导数为 $-4/3$ 。请说明因子 2、输入 x_i 和除以 3 各自来自哪里。

Notebook 会调用 `mean_square_loss` 与 `loss_derivative` 核对这两个数，再调用 `exact_checks` 用 `Fraction` 重算有理数结论。后者还检查完成平方公式：

$$L(w) = \frac{14}{3} \left(w - \frac{23}{14} \right)^2 + \frac{1}{14}.$$

所以最优参数为 $23/14$ ，最小损失为 $1/14$ 。用绝对容差检查该点导数接近 0；不要拿 0 当分母计算相对误差。

三 比较一次有限变化与切线预测

从 $w = 3/2$ 向右改变 $h = 1/100$ 。线性预测的损失变化为 $-1/75$ ，而实际变化为 $-193/15000$ ，新的损失为 $769/5000$ 。两种变化的差为 $(14/3)h^2$ 。

在 Notebook 中分别显示旧损失、新损失、线性预测变化和实际变化。数值不一致并非导数公式错了，因为导数只给出局部一阶响应。再把步长换成 $h = 1/2$ ，观察负导数为什么没有保证这次有限更新下降。

每次只改变一个设置并记录；同时换数据、损失和步长，无法定位差异原因。

四 用立方函数区分截断与舍入

取 $f(x) = x^3$ ，考察点 $x = 1$ ，真实导数为 3。前向与中心差分在精确算术下为

$$D_h f(1) = 3 + 3h + h^2, \quad C_h f(1) = 3 + h^2.$$

先用 `Fraction` 对 $h = 1/20$ 检查精确误差，结果分别为 $61/400$ 与 $1/400$ 。随后扫描从 10^{-1} 到 10^{-17} 的步长，显示浮点估计、绝对误差、真实浮点采样位移和状态。

中心差分使用名义分母 $2h$ 。因为加减法也在浮点数中执行，实际正负位移未必逐位相等，报告中特意单独保存 `actual_positive_step` 与 `actual_negative_step`。不应把名义 h 当成机器中的精确位移证明。

误差图的横纵轴均为对数轴。严格为 0 的误差无法取对数，图会明确说明这类点没有进入对数曲线；不把 0 替换成一个虚构的观测值。被拒绝的步长也不参与误差折线，但原始表保留失败原因。

观察较大步长时的下降趋势与极小步长时的停滞、起伏或失败。不能预设每个点都排成光滑的 U 形，也不要要求不同机器上的最低点在同一个步长。你要解释两种误差机制，而不是把当前机器的最佳 h 背成定律。

五 让错误有明确名字

Notebook 中用 `try` 和 `except` 展示预期失败。函数的非法输入或数学定义域越界必须保留原因，不能用返回 0 掩盖。

第一组检查不可导。对绝对值在 0 处，前向为 1，后向为 -1，中心为 0；对 $\max(0, x)$ ，左右为 0 和 1，中心为 1/2。请写明：这些中心值是对称差商，不是已经存在的两侧导数。

第二组检查定义域。对 `math.log`，取 $a = 0.05, h = 0.1$ ，中心差分会采到负数，必须失败。换成 $h = 0.01$ 后，三个采样点才都在正定义域内。不能在函数内部偷偷取绝对值来让程序继续跑。

第三组检查机器分辨率。对 $a = 10^{20}, h = 0.5$ 或 $a = 1, h = 10^{-20}$ ，采样点可能与起点相同。包装函数明确拒绝，避免把“没有成功移动”误报成“变化率为 0”。

第四组检查接口。步长须有限且严格为正；输入和函数输出须为有限实数标量，不能用 `True`、数字字符串、复数、NaN 或无穷大代替。CSV 的字符串解析是明确的数据读取步骤，核心数学函数不做这种隐式转换。

六 重跑与验收

最后调用 `boundary_checks` 与完整 `run`。预期通过 205 项精确检查和 35 项边界及基本结果检查。测试显式抛出异常，不依赖会被 `python -O` 关闭的 `assert` 语句；已额外在优化模式下验证。测试数量代表当前教学程序的具体检查，不是数学结论的证明数量。

输出 `report.json` 保存数据摘要、版本、手算值、精确检查、边界结果和步长扫描。`difference_scan.csv` 保存两种函数、两种公式、17个步长，共68条尝试；失败也占一条记录，不能为了画漂亮曲线把失败从原始结果中删除。

主验收入口要求原始三条基准数据。若更改 CSV，入口会拒绝，以免仍使用旧手算值验收新问题。探索新数据时，可以显式调用通用损失函数，并另建基准、报告和数据说明。保持原始数据文件以便重现本讲。

在新终端重跑脚本，再清空内核重跑 Notebook。同环境脚本结果应相同；跨平台须结合容差和版本，不能普遍要求字节一致。

提交一份短实验记录：一项完整手算，一张有轴标注的误差曲线，一项不可导反例，一项定义域或分辨率失败，以及你实际使用的环境版本。解释每个证据支持什么、没有支持什么，就完成了本实验。